

-

CONFIDENTIAL — NDA REQUIRED — SMEPRO TECHNOLOGIES — INTERNAL
ENGINEERING USE ONLY

-

IOS+ Engineering Body — Document 5

RAG Vault Retrieval Pipeline: UCO-Partitioned Knowledge Architecture

Organization:

SMEPro Technologies

Classification:

CONFIDENTIAL — NDA REQUIRED

Document Series:

IOS+ Engineering Body (EB)

Version:

1.0

Document ID:

EB-DOC-005

Date:

May 2026

Status:

Production Specification

Layer:

YBR Layer 5 (L5) — RAG Vault

Cross-References:

EB Doc 1 (Architecture) | EB Doc 2 (Cryptographic Audit) | EB Doc
3 + Amendment v1.1 (COS+ Schema) | EB Doc 4 (Gate 530 UCO Dimension
Matrix) | EB Doc 6 (Deployment Runbook)

NDA Notice

This document contains proprietary architecture specifications for the IOS+ platform and the RAG Vault Retrieval Pipeline. Distribution is strictly limited to personnel with executed SMEPro Technologies Non-Disclosure Agreements. Unauthorized reproduction, disclosure, or transmission is prohibited.

Table of Contents

- 1. Executive Summary
 - 1.1 Scope and Purpose
- 2. UCO Sector Partition Architecture
 - 2.1 The 20-Partition Model
 - 2.2 Partition-Level pgvector Configuration
 - 2.3 UCO Node Vector Embeddings in uco_nodes
- 3. NAICS Decoder-Driven Routing
 - 3.1 The NAICS Full Decoder Role
 - 3.2 Routing Algorithm
 - 3.3 Multi-Sector Query Fan-Out
- 4. UCO Node ID as Retrieval Dimension

- 4.1 Chunk Metadata Schema — UCO Fields
- 4.2 UCO Tagging During Ingestion
- 4.3 UCO-Aware Retrieval Query
- 5. Agency Registry as Source Authority Scoring
 - 5.1 The Authority Hierarchy
 - 5.2 Authority Score Computation
 - 5.3 Top Agency Coverage
- 6. Filing Calendar Integration (Temporal Relevance)
 - 6.1 Temporal Decay in RAG Retrieval
 - 6.2 Temporal Relevance Algorithm
 - 6.3 Overdue Filing Detection
- 7. Compliance Chain Re-Ranking
 - 7.1 The Compliance Chain as Re-Ranking Signal
 - 7.2 Chain Matching Algorithm
 - 7.3 Final Retrieval Score
- 8. Tenant-Scoped Query Routing End-to-End
 - 8.1 Full Retrieval Pipeline
 - 8.2 Performance Budget for RAG Vault
- 9. Knowledge Ingestion Pipeline
 - 9.1 Ingestion Architecture
 - 9.2 Ingestion Steps
 - 9.3 Agency Corpus Coverage Goals
- 10. Cross-Cutting Regulations Handling
 - 10.1 The XSC Universal Partition
 - 10.2 XSC Retrieval Integration
- 11. Performance Specifications
 - 11.1 Throughput and Latency
 - 11.2 Scaling Model
- 12. Monitoring and Observability
 - 12.1 Key Metrics
 - 12.2 Alerting Thresholds
- 13. Engineering Body Cross-References

1. Executive Summary

1.1 Scope and Purpose

The **RAG Vault** is the retrieval-augmented generation knowledge layer within the IOS+ platform, operating at **YBR Layer 5 (L5)**. Prior Engineering Body specifications described the RAG Vault as a generic vector store. This document defines the **production architecture**: RAG Vault is a **UCO-partitioned knowledge graph** in which all regulatory knowledge is organized, indexed, and retrieved along **20 sector partitions** corresponding to the 19 NAICS industry sectors and 1 universal cross-cutting partition (**UCO-XSC**), derived from the Universal Compliance Decoding Matrix.

Every document chunk ingested into RAG Vault is tagged with applicable **UCO_NODE_IDS** from the **uco_nodes** table (EB Doc 3 §6.2). Every retrieval query is first resolved to a sector partition via the tenant's NAICS profile, then executed within the applicable partition(s) plus the universal XSC partition. Retrieved chunks feed **Gate 530** with UCO-contextualized evidence, enabling the compliance evaluation defined in EB Doc 4 to ground its decisions in authoritative regulatory source documents.

Architecture Position

RAG Vault is not a generic document store. It is a compliance-domain knowledge graph whose retrieval semantics are governed entirely by the UCO node graph, NAICS sector taxonomy, and agency authority hierarchy. All retrieval decisions are traceable to the Universal Compliance Decoding Matrix.

This document covers the following architectural domains:

- **UCO Sector Partition Architecture** — The 20-partition pgvector model and HNSW index configuration
- **NAICS Decoder-Driven Routing** — How tenant NAICS profiles drive partition selection

- **UCO Node ID as Retrieval Dimension** — Chunk metadata schema, tagging, and UCO-aware retrieval queries
 - **Agency Registry as Source Authority Scoring** — Authority hierarchy and score computation
 - **Filing Calendar Integration** — Temporal relevance and overdue filing detection
 - **Compliance Chain Re-Ranking** — Lineage-aware scoring boost
 - **End-to-End Query Routing** — Full pipeline from gRPC request to Evidence Package
 - **Knowledge Ingestion Pipeline** — Document processing, UCO tagging, and corpus coverage
 - **Cross-Cutting Regulations** — XSC universal partition handling
 - **Performance, Monitoring, and Observability** — SLA targets and alerting
-

2. UCO Sector Partition Architecture

2.1 The 20-Partition Model

RAG Vault maintains **20 logical partitions**, each corresponding to a sector in the Universal Compliance Decoding Matrix. Partitions are implemented as separate `pgvector` table partitions within COS+, using PostgreSQL table partitioning on the `sector_code` column of `rag_chunks`. Refer to EB Doc 3 §6.10, table: `rag_vault_sector_partitions`.

Each partition carries its own HNSW vector index, calibrated to that sector's knowledge density. Risk tier designations (CRITICAL / HIGH / MEDIUM) align directly with the UCO node risk weight distributions defined in EB Doc 4.

Partition Name	Code	Sector	NAICS Divs.	UCO Nodes	Risk Tier	Table Partition
<code>rag_chunk_s_energy</code>	01	Energy	21, 22,	54	CRITICAL	<code>rag_chunk_s_p01</code>

Partition Name	Code	Sector	NAICS Divs.	UCO Nodes	Risk Tier	Table Partition
			486		L	
rag_chunk_s_healthcare	02	Healthcare & Social Assistance	62	36	CRITICAL	rag_chunk_s_p02
rag_chunk_s_finance	03	Finance, Banking, Securities	52	30	CRITICAL	rag_chunk_s_p03
rag_chunk_s_fooddrug	04	Food, Drug & Agriculture	31, 32, 11	16	HIGH	rag_chunk_s_p04
rag_chunk_s_mfg	05	Manufacturing & Transportation	31-33, 48-49	27	HIGH	rag_chunk_s_p05
rag_chunk_s_telecom	06	Telecom, Environment & Defense	51, 56, 92	20	HIGH	rag_chunk_s_p06
rag_chunk_s_insurance	07	Insurance (All Lines)	524, 525	35	CRITICAL	rag_chunk_s_p07
rag_chunk_s_realestate	08	Real Estate & Construction	23, 53	10	HIGH	rag_chunk_s_p08
rag_chunk_s_agriculture	09	Agriculture, Forestry, Fishing	11	8	MEDIUM	rag_chunk_s_p09
rag_chunk_s_mining	10	Mining & Quarrying	212	5	HIGH	rag_chunk_s_p10
rag_chunk_s_trade	11	Wholesale & Retail Trade	42, 44-45	15	HIGH	rag_chunk_s_p11
rag_chunk_s_professional	12	Professional Services	54	13	HIGH	rag_chunk_s_p12
rag_chunk_s_education	13	Education	61	10	CRITICAL	rag_chunk_s_p13

Partition Name	Code	Sector	NAICS Divs.	UCO Nodes	Risk Tier	Table Partition
on					L	
rag_chunk_s_arts	14	Arts & Entertainment	71, 72	9	HIGH	rag_chunk_s_p14
rag_chunk_s_accommodation	15	Accommodation & Food Services	72	10	MEDIUM	rag_chunk_s_p15
rag_chunk_s_admin	16	Admin & Waste Services	56	9	HIGH	rag_chunk_s_p16
rag_chunk_s_other	17	Other Services	81	9	HIGH	rag_chunk_s_p17
rag_chunk_s_publicadministration	18	Public Administration	92	9	CRITICAL	rag_chunk_s_p18
rag_chunk_s_mgmt	19	Management of Companies	55	6	HIGH	rag_chunk_s_p19
rag_chunk_s_xsc	XSC	Cross-Cutting (Universal)	ALL	19	CRITICAL	rag_chunk_s_xsc

① Partition Totals

Total UCO nodes across all sector partitions: **320 nodes**. The XSC partition's 19 nodes are universal and are duplicated logically into all tenant-scoped retrieval contexts. Energy (54 nodes) is the highest-density sector and receives the most aggressive HNSW parameters. See §2.2.

2.2 Partition-Level pgvector Configuration

Each sector partition maintains its own **HNSW index** calibrated to the knowledge density of that sector. HNSW parameters `m` (number of bi-directional links), `ef_construction` (build-time search width), and `ef_search` (query-time search width) are tuned per tier:

Tier Group	Sectors	m	ef_construction	ef_search
CRITICAL — High Density	Energy (01), Healthcare (02), Insurance (07)	32	256	128
CRITICAL/ HIGH — Medium Density	Finance (03), Manufacturing (05), Telecom (06)	24	200	100
HIGH/MEDIUM — Standard	All remaining sectors (04, 08-19)	16	128	64
XSC — Universal	Cross-Cutting (XSC)	16	128	64

The following DDL template is used for HNSW index creation. The Energy partition is shown as the reference example:

```
-- Example: Energy partition HNSW index CREATE INDEX CONCURRENTLY
idx_rag_chunks_p01_embedding ON rag_chunks_p01 USING hnsw (embedding
vector_cosine_ops) WITH (m = 32, ef_construction = 256); SET
hnsw.ef_search = 128; -- applied at session/query level
```

⚠ Index Build Note

All HNSW indexes must be created using `CREATE INDEX CONCURRENTLY` to avoid table locks during initial corpus load. Index build time for the Energy partition at full corpus volume is estimated at 8-14 minutes. See EB Doc 6 (Deployment Runbook) for the seed sequence and index build ordering.

2.3 UCO Node Vector Embeddings in `uco_nodes`

Each UCO node row in the `uco_nodes` table carries a `vector_embedding` column (`vector(1536)`) storing an **OpenAI text-embedding-3-small** embedding. The input text is the concatenation of the node's key fields:

```
embedding_input = regulation_name || ' ' || specific_activity || ' ' || cfr_usc_citation
```



```
|| '' || penalties_consequences
```

This enables **semantic UCO node lookup** to find the most relevant compliance obligations for a given query, independent of exact NAICS code matching. The HNSW index configuration for `uco_nodes` is:

- Index type: HNSW on `uco_nodes.vector_embedding`
- Distance metric: `vector_cosine_ops`
- Parameters: `m=16`, `ef_construction=128`
- `ef_search`: 64 (at query time)

3. NAICS Decoder-Driven Routing

3.1 The NAICS Full Decoder Role

The `naics_decoder` table (EB Doc 3 §6.4) maps every 3-digit NAICS subsector to its primary governing agencies, applicable UCO node count, risk tier, and COS+ sector sheet. The NAICS decoder is RAG Vault's **routing table**: a query arrives with tenant NAICS codes → decoder resolves subsector → identifies sector partition → routes retrieval to the correct `rag_chunks` partition(s).

The decoder resolution chain is:

1. Tenant provides NAICS codes at onboarding (stored in `tenant_naics_profiles`)
2. Each NAICS code is truncated to the 3-digit subsector level
3. The `naics_decoder` table resolves each subsector to a `cos_plus_sheet` (sector code 01-19)
4. Sector codes map deterministically to the partition table names via `SECTOR_PARTITION_MAP`
5. The XSC partition is always appended to the resolved partition list

3.2 Routing Algorithm

The following TypeScript function implements partition resolution. It is executed at the start of every retrieval request within the `rag-vault` service:

```
async function resolveRAGPartitions( tenantId: UUID, queryContext:
QueryContext ): Promise<string[]> {    // 1. Fetch tenant NAICS profile
const profile = await db.query(        `SELECT naics_codes, active_sectors
FROM tenant_naics_profiles        WHERE tenant_id = $1`,        [tenantId]    );
// 2. Map each NAICS code to 3-digit subsector    const subsectors =
profile.naics_codes.map(        (code: string) => code.substring(0,
3)    );    // 3. Resolve sector codes from naics_decoder    const sectorRows =
await db.query(        `SELECT DISTINCT cos_plus_sheet        FROM naics_decoder
WHERE naics_3dig = ANY($1)`,        [subsectors]    );    const activeSectors =
sectorRows.map(        (r: any) => r.cos_plus_sheet    );    // 4. Always include
XSC cross-cutting partition    const partitions = [...activeSectors, 'XSC'];
// 5. Map sector codes to partition table names    return
partitions.map(        (s: string) => `rag_chunks_${
{SECTOR_PARTITION_MAP[s]}`    ); }
```

3.3 Multi-Sector Query Fan-Out

When a tenant operates across multiple NAICS sectors (e.g., a hospital system with real estate holdings and internal HR operations), the query is **fanned out across all applicable partitions in parallel** using PostgreSQL parallel query workers. Results are merged and re-ranked by UCO relevance score before being returned to the L5 synthesis layer.

① Maximum Fan-Out

Maximum fan-out is all 20 partitions. This scenario is rare and occurs only for pan-industry tenants such as large government contractors or diversified holding companies. The all-20-partition P99 latency target is < 35ms. See §11.1 for the full performance specification.

4. UCO Node ID as Retrieval Dimension

4.1 Chunk Metadata Schema — UCO Fields

Every chunk stored in `rag_chunks` carries UCO metadata columns in addition to the base schema from EB Doc 3. These columns are added by the **V4 migration** (EB Doc 3 §6.11):

Column Name	Type	Description
<code>applicable_uco_node_ids</code>	<code>TEXT[]</code>	Array of <code>UCO_NODE_IDs</code> for regulations directly evidenced by this chunk. GIN-indexed for overlap queries.
<code>primary_sector_code</code>	<code>VARCHAR(4)</code>	Single sector this chunk primarily serves; matches partition key column.
<code>agency_code</code>	<code>VARCHAR(16)</code>	Governing agency whose regulatory text this chunk derives from. References <code>agency_registry.agency_code</code> .
<code>cfr_citation</code>	<code>VARCHAR(256)</code>	CFR/USC citation if the chunk is sourced from regulatory text.
<code>jurisdiction_level</code>	<code>VARCHAR(64)</code>	Jurisdictional scope: Federal / State / Local / International.
<code>risk_weight_max</code>	<code>SMALLINT</code>	Maximum <code>risk_weight</code> across all <code>applicable_uco_node_ids</code> . Used as ranking boost factor.
<code>filing_frequency</code>	<code>VARCHAR(64)</code>	Filing frequency from the applicable UCO node; enables temporal relevance scoring.
<code>source_authority_score</code>	<code>NUMERIC(4,3)</code>	Pre-computed source authority score. See §5. Range: 0.000–1.200 (may exceed 1.0 with citation

Column Name	Type	Description
		density boost).

4.2 UCO Tagging During Ingestion

When a new document is ingested into RAG Vault, the ingestion pipeline executes the following UCO tagging procedure for every chunk:

6. **Chunking:** Text is split using `RecursiveCharacterTextSplitter` at 512-token chunk size with 64-token overlap.
7. **Embedding:** Each chunk is embedded via `text-embedding-3-small` (1536 dimensions).
8. **UCO Tagger:** The chunk embedding is compared against all rows in `uco_nodes.vector_embedding` via cosine similarity. Any UCO node with similarity > 0.82 is tagged as applicable and added to `applicable_uco_node_ids`.
9. **Sector Assignment:** `primary_sector_code` is set to the `sector_code` of the highest-ranked (highest similarity) matching UCO node.
10. **Partition Routing:** The chunk is inserted into the corresponding `rag_chunks_p{sector_code}` partition.
11. **Metadata Population:** All UCO metadata columns are populated including `risk_weight_max`, `filing_frequency`, and `source_authority_score`.

⚠ UCO Similarity Threshold

The UCO tagging threshold of 0.82 is tuned for precision over recall. Lowering this threshold risks tagging chunks with inapplicable compliance obligations, which would corrupt the compliance evidence chain. Any threshold change requires a full corpus re-embedding and must be reviewed by the IOS+ architecture team.

4.3 UCO-Aware Retrieval Query

The following SQL query implements UCO-aware retrieval within a single sector partition. The example targets the Energy partition (rag_chunks_p01). Parameters: \$1 = query embedding vector, \$2 = tenant's active UCO node ID array:

```
-- UCO-aware retrieval within the energy sector partition
SELECT
  rc.chunk_id, rc.source_document_id, rc.chunk_text,
  rc.applicable_uco_node_ids, rc.agency_code, rc.cfr_citation,
  rc.risk_weight_max, rc.source_authority_score,
  1 - (rc.embedding <=> $1::vector) AS cosine_similarity,
  boost: multiply by (1 + 0.1 * risk_weight_max / 10) (1 - (rc.embedding <=>
  $1::vector)) * (1 + 0.1 * rc.risk_weight_max / 10.0) AS
  uco_relevance_score
FROM rag_chunks_p01 rc
WHERE
  rc.applicable_uco_node_ids && $2
  AND 1 - (rc.embedding <=> $1::vector) > 0.75
ORDER BY uco_relevance_score DESC
LIMIT 20;
```

GIN Index Note

The && (overlap) operator on applicable_uco_node_ids uses a GIN (Generalized Inverted Index) created on rag_chunks_p{n}.applicable_uco_node_ids. This pre-filters the HNSW scan to only UCO-relevant chunks before cosine similarity is evaluated, significantly reducing scan cardinality for large partitions.

5. Agency Registry as Source Authority Scoring

5.1 The Authority Hierarchy

The agency_registry table (EB Doc 3 §6.3) encodes 80+ federal agencies with their agency_type. Source authority scoring uses agency_type to assign a **base authority score** to each knowledge chunk at ingestion time.

Agency Type	Base Authority Score	Example Agencies
Cabinet Department (direct)	0.95	DOJ, HHS, DOE, DOL, DOT

Agency Type	Base Authority Score	Example Agencies
Independent Agency (primary regulator)	0.90	SEC, FCC, CFPB, NRC, FTC, FDIC
Cabinet Sub-Agency (operational)	0.85	FDA, OSHA, EPA-OECA, DEA, CMS
State Regulatory Body	0.80	State DOIs, State Banking Departments, State AGs
SRO (Self-Regulatory Organization)	0.75	FINRA, PCAOB, CBOT
Government Corporation	0.70	FDIC (legacy), PBGC
Industry Standard Body	0.65	NAIC, ISO, ANSI
Academic / Third-Party	0.50	Law reviews, compliance consultants

5.2 Authority Score Computation

The `source_authority_score` is computed at ingestion time and stored on the chunk record. It is a composite of the base authority score, jurisdictional scope, document recency, and citation density within the chunk text:

```

source_authority_score =  base_authority_score  × jurisdiction_multiplier  --
Federal=1.0 | State=0.9 | Local=0.8 | International=0.85  × recency_multiplier
--
<
1yr=1.0 | 1–3yr=0.9 | 3–5yr=0.75 |
>
5yr=0.60  × citation_density          -- proportion of chunk text with explicit CFR/USC
citations                                -- (range: 0.80–1.20; a fully cited chunk receives 1.20x
boost)

```

① Pre-Computation Note

The `source_authority_score` is pre-computed at ingestion and stored as `NUMERIC(4,3)`. It is used as a secondary ranking signal in the UCO-aware retrieval query (§4.3) and in the final composite retrieval score formula (§7.3). Scores must be recomputed if a document's recency multiplier bucket changes on the annual corpus refresh cycle.

5.3 Top Agency Coverage (by UCO Node Count)

The 15 most-represented agencies by UCO node count. Each agency's regulatory corpus is the primary knowledge source for its associated UCO nodes and dominates the corresponding sector partition's knowledge base.

Rank	Agency	UCO Nodes	Primary Regulatory Corpus
1	EPA	17	CFR Title 40 (RCRA, TSCA, CAA, CWA)
2	OSHA	8	29 CFR Part 1910, Part 1926, VPP guidance
2	State DOIs	8	State insurance regulations (all 50 states)
4	FTC	7	16 CFR consumer protection rules, FTC Act guidance
5	FERC	6	18 CFR energy regulations, tariff filings
5	FDA-CDRH	6	21 CFR medical device regulations
5	FDA-CFSAN	6	21 CFR food safety regulations
5	EPA/State DEQ	6	State environmental quality regulations
9	CMS	5	42 CFR Parts 400–

Rank	Agency	UCO Nodes	Primary Regulatory Corpus
			499 (Medicare/Medicaid)
9	FDA-CDER	5	21 CFR pharmaceutical regulations
9	FinCEN	5	31 CFR BSA/AML regulations
9	CFPB	5	12 CFR consumer financial protection rules
9	FCC	5	47 CFR telecommunications regulations
14	NRC	4	10 CFR nuclear regulatory requirements
14	DEA	4	21 CFR controlled substance regulations

6. Filing Calendar Integration (Temporal Relevance)

6.1 Temporal Decay in RAG Retrieval

Compliance knowledge has an inherent **temporal dimension**: a regulation's filing deadline, effective date, or amendment date determines whether a retrieved chunk is currently actionable. The `filing_calendar` table (EB Doc 3 §6.9) drives temporal relevance scoring by associating each UCO node with its next due date, filing frequency, and current filing status for each tenant.

Temporal relevance is applied as a **post-retrieval multiplier** on the base cosine similarity score. It does not alter the HNSW vector search itself — it re-weights results after candidate chunks have been retrieved.

6.2 Temporal Relevance Algorithm

The temporal score is computed per-chunk at retrieval time based on the `next_due_date` in `filing_calendar` for the tenant × UCO node intersection:

```
temporal_score = base_similarity × deadline_proximity_multiplier  
deadline_proximity_multiplier:  filing_status = 'OVERDUE'      → 1.50  
(CRITICAL — always surface at top)  next_due_date within 30 days    → 1.25  
(URGENT — elevate in results)    next_due_date within 31–90 days  → 1.10  
(APPROACHING)  next_due_date > 90 days or no date → 1.00 (BASELINE)
```

6.3 Overdue Filing Detection

The `filing_calendar` partial index on `filing_status = 'OVERDUE'` (EB Doc 3 §6.9) enables a fast **pre-retrieval check**. Before any vector search is initiated:

12. Gate 530 queries `filing_calendar` for any `OVERDUE` filings scoped to the current tenant
13. If any UCO node is `OVERDUE`, those nodes are **injected as mandatory context** into the retrieval query, bypassing cosine similarity thresholds
14. `OVERDUE` chunks receive the 1.50× temporal multiplier and are pinned to the top of the final result set

⚠ Critical — Overdue Filing Guarantee

`OVERDUE` compliance obligations must **never** be buried by relevance scoring. The mandatory injection mechanism ensures that overdue filings appear in every Evidence Package regardless of query context. Monitoring for overdue injections is specified in §12.2.

7. Compliance Chain Re-Ranking

7.1 The Compliance Chain as Re-Ranking Signal

The `compliance_chains` table (EB Doc 3 §6.6) stores the full regulatory lineage for each UCO node in the format:

```
CIP:{code} → NAICS:{code} → {JurisdictionLevel} → {Agency} →  
{RegulationName}
```

When RAG Vault retrieves chunks, the compliance chain provides a structured re-ranking signal: chunks whose `compliance_chain_ref` exactly or partially matches the tenant's resolved compliance chain are boosted. This ensures that jurisdictionally and operationally relevant regulations are surfaced above generic regulatory text for a given tenant profile.

7.2 Chain Matching Algorithm

```
function chainMatchScore( chunkUcoNodes: string[], tenantChain:  
ComplianceChain ): number {    const matchingNodes =  
chunkUcoNodes.filter(nodeId =>  
tenantChain.applicableNodes.includes(nodeId)    );    // Boost = (matching  
nodes / total chunk UCO nodes)    // *  
max_risk_weight_of_matching_nodes / 10    const chainBoost =  
(matchingNodes.length / chunkUcoNodes.length) *  
(Math.max(    ...matchingNodes.map(n => uco_node_risk_weights[n])    ) /  
10);    return Math.min(chainBoost, 1.0); // cap boost at 1.0 }
```

7.3 Final Retrieval Score

The final composite retrieval score combines all ranking signals. This is the score used to order the merged multi-partition result set before top-K selection:

```
final_score =    cosine_similarity × (1 + 0.15 × source_authority_score) --  
authority boost:    max +15% × (1 + 0.10 × chain_match_score)    -- chain  
match boost:    max +10% × deadline_proximity_multiplier    -- temporal  
urgency:    up to +50% × (1 + 0.05 × risk_weight_max / 10)    -- risk weight boost:
```

max +5%

Boost Component	Max Contribution	Source
Source authority boost	+15%	source_authority_score from agency_registry
Compliance chain match boost	+10%	compliance_chains lineage matching
Temporal urgency multiplier	+50%	filing_calendar.filing_status / next_due_date
Risk weight boost	+5%	risk_weight_max from uco_nodes

8. Tenant-Scoped Query Routing End-to-End

8.1 Full Retrieval Pipeline

The following is the complete step-by-step retrieval pipeline from gRPC request receipt to Evidence Package delivery:

Step 1 — Receive Retrieval Request

Receive retrieval request from L5 RAG Vault gRPC interface as defined in EB Doc 1, \$RAGVaultService.QueryKnowledge. Input: tenant_id, query text, query context object.

Step 2 — Load Tenant NAICS Profile

Load tenant NAICS profile from tenant_naics_profiles (primary key lookup on tenant_id). Target: < 2ms. Cached in application-layer LRU cache with 60-second TTL.

Step 3 — Overdue Filing Check

Query `filing_calendar` for OVERDUE items scoped to the tenant using the partial index on `filing_status = 'OVERDUE'`. Target: < 5ms. Inject mandatory UCO nodes if any OVERDUE items are found.

Step 4 — Partition Resolution

Resolve applicable partitions via NAICS decoder (§3.1–3.2). Output: list of partition table names including XSC.

Step 5 — Parallel Fan-Out

Fan out vector searches across all applicable partitions in parallel using PostgreSQL parallel query workers. Each partition search is an independent query submitted concurrently.

Step 6 — UCO-Aware Retrieval Per Partition

For each partition, execute the UCO-aware retrieval query (§4.3): GIN filter on `applicable_uco_node_ids`, cosine similarity threshold > 0.75, ORDER BY `uco_relevance_score` DESC, LIMIT 20 per partition.

Step 7 — Merge and Re-Rank

Merge results from all partitions. Apply temporal multiplier (§6.2) and compliance chain boost (§7.2) to compute `final_score` for each candidate chunk. Target: < 10ms for merge + re-rank.

Step 8 — Top-K Selection

Select top-K = 10 chunks by `final_score` from the merged and re-ranked result set. OVERDUE-injected chunks are pinned and count toward top-K.

Step 9 — Build Evidence Package

Construct the Evidence Package: include `chunk_text`, `source_document_id`, `applicable_uco_node_ids`, `cfr_citation`, `agency_code`, `risk_weight_max`, `source_authority_score`, `final_score`. Target: < 3ms.

Step 10 — Return to L5 / Gate 530

Return Evidence Package to L5. L5 synthesizes the tenant response and passes the Evidence Package to Gate 530 at L4 for compliance evaluation per EB Doc 4.

8.2 Performance Budget for RAG Vault

RAG Vault operates within the IOS+ end-to-end P99 latency budget of **300ms** for the full query lifecycle. The RAG Vault budget allocation is 40ms P99:

Pipeline Step	Target Latency	Notes
Tenant NAICS profile lookup (Step 2)	< 2ms	Primary key lookup; LRU cached
Filing calendar overdue check (Step 3)	< 5ms	Partial index on <code>filing_status = 'OVERDUE'</code>
Vector search per partition (Steps 5-6)	< 20ms	HNSW <code>ef_search</code> optimized per sector; parallel fan-out
Fan-out merge + re-ranking (Step 7)	< 10ms	In-memory sort across merged candidate sets
Evidence Package construction (Steps 8-9)	< 3ms	Struct assembly; no additional DB calls
Total RAG Vault P99 (end-to-end)	< 40ms	Full pipeline including routing, ranking, packaging

9. Knowledge Ingestion Pipeline

9.1 Ingestion Architecture

The RAG Vault ingestion pipeline runs as a separate service (`rag-ingestion-worker`) within the IOS+ monorepo. It processes source documents from the `rag_sources` table (EB Doc 3 base schema) and produces chunks in the 20 sector-partitioned `rag_chunks` tables. The ingestion worker is stateless and horizontally scalable — additional pods are added to increase throughput during large corpus loads.

9.2 Ingestion Steps

Step 1 — Document Source Registration

New regulatory document registered in `rag_sources` with: `source_type` (CFR_TEXT | AGENCY_GUIDANCE | COURT_OPINION | INDUSTRY_STANDARD), `governing_agency_code` (references `agency_registry`), `applicable_naics_codes`, `applicable_uco_node_ids` (human curator provides initial list for authority validation).

Step 2 — Text Extraction

PDF / HTML / DOCX parsing → normalized UTF-8 text. Parser: Apache Tika (PDF/DOCX), html-to-text (HTML). Character normalization: Unicode NFC form, whitespace normalization, header/footer stripping for CFR documents.

Step 3 — Chunking

`RecursiveCharacterTextSplitter`: `chunk_size=512` tokens, `overlap=64` tokens. Boundary-aware: respects paragraph and section breaks in CFR formatted text. Chapter headings and 26 USC section markers are treated as hard boundaries.

Step 4 — Embedding Generation

text-embedding-3-small (1536 dimensions) via OpenAI API. Batched at 100 chunks per API call. Retry policy: exponential backoff, max 3 retries, circuit breaker at 5 consecutive failures.

Step 5 — UCO Tagger

For each chunk, compute cosine similarity against all rows in `uco_nodes.vector_embedding` via HNSW index. Threshold: 0.82. All matching nodes added to `applicable_uco_node_ids`.

Step 6 — Sector Assignment

`primary_sector_code` set to the `sector_code` of the highest-cosine-similarity UCO node. Determines target partition for INSERT.

Step 7 — Authority Score Computation

Compute `source_authority_score` per the formula in §5.2. Requires: `agency_type` from `agency_registry`, `document_date` from `rag_sources`, CFR citation density regex scan, `jurisdiction_level`.

Step 8 — Temporal Metadata

Extract and parse due dates from chunk text using CFR deadline regex patterns. Cross-reference against `filing_calendar` to validate and populate `filing_frequency` for matched UCO nodes.

Step 9 — Partition Routing and Insert

INSERT INTO `rag_chunks` PARTITION (`rag_chunks_p{sector_code}`) with all metadata columns populated. Uses COPY protocol for bulk inserts (> 1000

chunks); single INSERT for incremental updates.

Step 10 — HNSW Index Update

PostgreSQL updates the HNSW index incrementally on INSERT. No manual reindex required for incremental ingestion. Full `REINDEX CONCURRENTLY` is scheduled quarterly for corpus-wide optimization.

9.3 Agency Corpus Coverage Goals

Target corpus coverage per agency from `agency_registry`, ordered by UCO node count:

Agency	UCO Nodes	Target Corpus Coverage
EPA	17	Full CFR Title 40 core chapters; RCRA, TSCA, CAA, CWA guidance documents
OSHA	8	29 CFR Part 1910 (General Industry), Part 1926 (Construction), VPP guidance
FTC	7	16 CFR consumer protection rules; FTC Act guidance documents
FERC	6	18 CFR energy regulations; tariff filings, NERC reliability standards
FDA-CDRH	6	21 CFR Parts 800-898 medical device regulations; QSR guidance
FDA-CFSAN	6	21 CFR Parts 100-199 food safety regulations; FSMA rules
CMS	5	42 CFR Parts 400-499 Medicare/Medicaid conditions of participation
FDA-CDER	5	21 CFR Parts 200-299

Agency	UCO Nodes	Target Corpus Coverage
		pharmaceutical regulations; cGMP guidance
FinCEN	5	31 CFR Parts 1010-1030 BSA/AML regulations; SAR filing guidance
CFPB	5	12 CFR consumer financial protection rules; TILA, RESPA, ECOA regulations
FCC	5	47 CFR telecommunications regulations; CPNI, EEO, broadband rules

10. Cross-Cutting Regulations Handling

10.1 The XSC Universal Partition

The `rag_chunks_xsc` partition stores all regulatory knowledge for the **19 UCO-XSC-5xxx universal cross-cutting obligations**. It is always included in every tenant's retrieval query regardless of sector, because these obligations apply universally to all employers and entities operating in the United States.

Knowledge sources for the XSC partition include:

Regulatory Domain	Primary Sources	UCO Nodes
EEOC Civil Rights (Title VII / ADA / ADEA)	EEOC enforcement manuals, 29 CFR Part 1601	UCO-XSC-5001, 5003
FLSA Wage & Hour	29 USC §201 et seq., 29 CFR Parts 510-794	UCO-XSC-5002
FMLA Leave Requirements	29 CFR Part 825	UCO-XSC-5007

Regulatory Domain	Primary Sources	UCO Nodes
OSHA General Duty Clause	29 USC §654 Section 5(a) (1), OSHA guidance	UCO-XSC-5004
IRS Employment Tax	IRS Pub 15, Pub 15-B, 26 USC §§3101-3128	UCO-XSC-5006
State Data Breach Notification	All 50 state data breach statutes (indexed by jurisdiction_level = 'State')	UCO-XSC-5008
ADA Title III Public Accommodation	28 CFR Part 36, ADA Title III guidance	UCO-XSC-5009
WARN Act	29 USC §2101, 20 CFR Part 639	UCO-XSC-5010
ERISA Employee Benefits	29 CFR Parts 2500-2999; DOL ERISA guidance	UCO-XSC-5011

10.2 XSC Retrieval Integration

The XSC partition is always searched with `ef_search = 64` and `top-K = 5` per query (smaller top-K, since cross-cutting regulations are less query-specific and more context-agnostic than sector-specific results). XSC results are merged with sector results before re-ranking.

⚠ **Mandatory Inclusion — High-Risk XSC Nodes**

XSC chunks from the following high-risk universal nodes receive a **mandatory inclusion flag**. They are always included in the final top-K regardless of cosine similarity score, when the query touches HR, payroll, or workplace safety context:

- **UCO-XSC-5002** — FLSA wage & hour (risk_weight = 10)
- **UCO-XSC-5004** — OSHA General Duty Clause (risk_weight = 9)
- **UCO-XSC-5006** — IRS employment tax (risk_weight = 10)

11. Performance Specifications

11.1 Throughput and Latency

Metric	Specification	Notes
Single-partition vector search P50	< 8ms	HNSW ef_search optimized per sector; warm index in shared_buffers
Single-partition vector search P99	< 20ms	With GIN pre-filter on UCO node IDs applied before HNSW scan
5-partition fan-out P99	< 25ms	Parallel query workers; typical multi-sector SME tenant scenario
All-20-partition fan-out P99	< 35ms	Maximum fan-out — pan-industry tenants only
Total RAG Vault P99 (end-to-end)	< 40ms	Full pipeline: routing, vector search, ranking, Evidence Package
UCO tagger per chunk (ingestion)	< 5ms	HNSW cosine similarity lookup against uco_nodes.vector_embedding
Ingestion throughput	> 500 chunks/min	Single rag-ingestion-worker pod at steady state

11.2 Scaling Model

RAG Vault scales horizontally via additional `rag_reader` role connection pool slots (per EB Doc 3 §3.x). Each additional `rag-ingestion-worker` pod adds approximately 500 chunks/min of ingestion throughput.

The HNSW indexes are **read-scalable**: multiple concurrent readers share the same in-memory index without locking contention. Write concurrency during HNSW index updates (on INSERT) does require short-duration index page locks, but these are sub-millisecond for incremental ingestion at production volume. Bulk initial corpus loads should be performed during off-peak hours using the `COPY` protocol followed by a single post-load `REINDEX CONCURRENTLY`.

Capacity Planning Reference

At the target corpus size of ~2.5M chunks across all 20 partitions, the Energy partition (rag_chunks_p01) is projected to hold approximately 380,000 chunks (highest density). With m=32 HNSW, this requires approximately 4.8GB RAM for the in-memory index. Ensure adequate shared_buffers and OS page cache allocation per EB Doc 6 (Deployment Runbook).

12. Monitoring and Observability

12.1 Key Metrics

The rag-vault service emits the following Prometheus metrics. All metrics are scraped by the IOS+ Prometheus instance and visualized in the RAG Vault Grafana dashboard (see EB Doc 6 for dashboard provisioning):

Metric Name	Type	Labels	Description
rag_retrieval_latency_ms	Histogram	tenant_id, partition	End-to-end retrieval latency per request; track P50/P95/P99 buckets
rag_chunks_returned_count	Counter	partition	Total chunks returned by partition; tracks retrieval volume per sector
uco_tagger_similarity_score_avg	Gauge	sector_code	Average UCO tagger cosine similarity at ingestion time; alerts if coverage degrades
rag_overdue_injections_total	Counter	uco_node_id	Counts mandatory OVERDUE node injections; compliance-critical

Metric Name	Type	Labels	Description
			signal
ingestion_chunks_per_minute	Gauge	partition	Real-time ingestion throughput per partition; detects pipeline stalls
source_authority_score_p50	Gauge	sector_code	Median source authority score across all chunks in partition; tracks corpus quality

12.2 Alerting Thresholds

Condition	Threshold	Alert Action	Severity
rag_retrieval_latency_ms P99	> 45ms	PagerDuty alert — SLA breach imminent	CRITICAL
uco_tagger_similarity_score_avg	< 0.65	Slack alert — UCO coverage degrading; re-embedding required	WARNING
rag_overdue_injections_total	> 5 in 1 hour	Compliance officer notification via email + Slack	HIGH
ingestion_chunks_per_minute	= 0 for > 10 min	PagerDuty alert — ingestion pipeline stalled	CRITICAL
source_authority_score_p50	< 0.70	Slack alert — corpus quality degrading; curator review required	WARNING

13. Engineering Body Cross-References

The following cross-references map RAG Vault architectural elements to their authoritative definitions in other Engineering Body documents:

EB Document	Relevant Sections / Elements	Dependency Type
EB Doc 1 <i>Architecture</i>	RAGVaultService gRPC interface definition (§RAGVaultService.Query Knowledge); rag_reader PostgreSQL role and connection pool configuration	CONSUMES — RAG Vault implements this interface
EB Doc 2 <i>Cryptographic Audit</i>	Evidence Package schema that RAG Vault populates; Evidence Package signing procedure and audit trail for retrieved chunks	PRODUCES — RAG Vault populates the Evidence Package
EB Doc 3 + Amendment v1.1 <i>COS+ Schema</i>	rag_sources, rag_chunks, rag_vault_sector_partitions, uco_nodes, agency_registry, filing_calendar, compliance_chains, naics_decoder, tenant_naics_profiles; V4 migration (§6.11) adding UCO metadata columns to rag_chunks	DEPENDS ON — All schema tables are defined here
EB Doc 4 <i>Gate 530 UCO Dimension Matrix</i>	Gate 530 consumes the RAG Vault Evidence Package as UCO-contextualized compliance evidence; UCO node risk weights and policy actions that drive retrieval priority and OVERDUE injection logic	PROVIDES TO — RAG Vault feeds Gate 530
EB Doc 6 <i>Deployment Runbook</i>	Deployment seed sequence for RAG Vault	REFERENCED BY — Deployment procedures

EB Document	Relevant Sections / Elements	Dependency Type
	partition initialization; initial corpus loading procedures; HNSW index build ordering; Grafana dashboard provisioning for RAG Vault metrics	for this document

Document Completeness Note

This document (EB Doc 5) is the definitive specification for RAG Vault retrieval architecture. It supersedes any generic vector store description found in prior Engineering Body drafts or internal wikis. All implementation decisions regarding partition structure, HNSW parameters, UCO tagging thresholds, authority scoring, and retrieval scoring must be governed by this specification.

CONFIDENTIAL — SMEPro Technologies — Engineering Body Document 5

RAG Vault Retrieval Pipeline: UCO-Partitioned Knowledge Architecture — Version 1.0
— May 2026

Available to qualified technical evaluators under enhanced NDA. Unauthorized reproduction or distribution is strictly prohibited.

IOS+ Engineering Body Series | EB-DOC-005 | YBR Layer 5 (L5)